University of Salerno

Hadoop MapReduce and Spark Exercises

Student Information:
Name: Ferraioli Chiara
Student ID: 0622702160
Email: c.ferraioli30@studenti.unisa.it
Course Channel: A-H

Lecturer:
Name: D'Aniello Giuseppe
Email: gidaniello@unisa.it

January 2024

Contents

1 MapReduce

1.0.1 Introduction to the problem

In this Hadoop MapReduce project, the objective was to analyze a comprehensive dataset from the FIFA World Cup, focusing specifically on calculating the number of goals scored by substitute players for each team. The dataset contained detailed records for each player, including information such as RoundID, MatchID, Team Initials, Coach Name, Line-up, Shirt Number, Player Name, Position, and Event.

1.0.2 Solution

In the Mapper phase, the primary goal was to filter and process records relevant to substitute players. This was accomplished by examining the 'Line-up' attribute in each record. Records where the 'Line-up' field was marked as 'N' indicated that the player was a substitute. The focus was then shifted to the 'Event' field to identify instances of goals or penalties scored by these substitute players. Each time a substitute player scored a goal ('Event' = 'G') or a penalty ('Event' = 'P'), a key-value pair was emitted. The key in this pair was the 'Team Initials', and the value was '1', signifying one goal or penalty scored by a substitute player. This approach ensured that the data relevant to the study's aim was accurately captured and prepared for the next phase of processing.

In the Reducer phase, the aim was to aggregate the data received from the Mapper phase. The process involved a summation of the goals and penalties scored by substitute players for each team. The Reducer iterated through the intermediate key-value pairs, grouping them based on the 'Team Initials'. For each unique team, the total count of goals and penalties attributed to substitute players was accumulated. This aggregation was crucial as it provided the final tally of substitute goals and penalties for each team, which was the primary metric of interest in this analysis.

The final output of the Reducer phase was a set of key-value pairs where the key was 'Team Initials' and the value was the total count of substitute goals and penalties. This output succinctly presented the results of the analysis, offering clear insights into the contribution of substitute players to their teams' scoring in the World Cup.

DriverWorldCup

```
package mapreduce;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

/**
 *
 * @author chiar
 */
public class DriverWorldCup {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) throws Exception{
        // TODO code application logic here
        if(args.length != 2){
            System.err.println("Usage: DriverWorldCup <input path> <output path>");
            System.exit(-1);
```

```
        }

        Path inputPath;
        Path outputPath;

        inputPath = new Path(args[0]);
        outputPath = a Path(args[1]);

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf);

        job.setJobName("WorldCup");

        FileInputFormat.addInputPath(job, inputPath);

        // Set path of the output folder for this job
        FileOutputFormat.setOutputPath(job, outputPath);

        // Specify the class of the Driver for this job
        job.setJarByClass(DriverWorldCup.class);

        job.setInputFormatClass(TextInputFormat.class);

        job.setOutputFormatClass(TextOutputFormat.class);

        job.setMapperClass(MapperWorldCup.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        job.setReducerClass(ReducerWorldCup.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

MapperWorldCup

```
package mapreduce;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

/**
 *
 * @author chiar
 */
public class MapperWorldCup extends Mapper<Object, Text, Text, IntWritable>{
    private final static IntWritable one = new IntWritable(1);

    @Override
    public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
        String[] fields = value.toString().split(",", -1);

        String team = fields[2];
```

```
        String events = fields[8];

        String line_up = fields[4];
        if(line_up.contentEquals("N")){
             for (int i = 0; i < events.length(); i++) {
                if (events.charAt(i) == 'G') {
                    // Ensure that 'G' is not preceded by 'O' to avoid counting 'OG'
                    if (i == 0 || events.charAt(i - 1) != 'O') {
                        context.write(new Text(team), one);
                    }
                } else if (events.charAt(i) == 'P') {
                     if (i == 0 || events.charAt(i - 1) != 'M') {
                        context.write(new Text(team), one);
                    }
                }
            }
        }
    }
}
```

ReducerWorldCup

```
package mapreduce;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

/**
 *
 * @author chiar
 */
public class ReducerWorldCup extends Reducer<Text, IntWritable, Text, IntWritable>{
    private IntWritable result = new IntWritable();

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,

        int sum = 0;
        for (IntWritable val : values) {
          sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

1.1 Output

The output from the MapReduce Program is represented in the file **players.txt**, which contains the initials of the team associated with the number of goals scored by the bench players of that team. The content of the file is:

```
BRA 1
DEN 2
FRA 1
FRG 2
HON 1
HUN 5
```

NED 1
POL 1
POR 1
SCO 1
SLV 1
TCH 1

2 Spark

2.0.1 Introduction to the problem

The task is to analyze a dataset containing information about football players who participated in World Cup matches. The dataset includes various fields such as the Round ID, Match ID, Team Initials, Coach Name, Line-up status, Shirt Number, Player Name, Position, and various Events (like goals, cards, substitutions, etc.).

The specific challenge in this exercise is to find the team (or teams, in case of ties) that has won the most matches with the fewest number of players on the field at the end of the game. This requires processing and analyzing the match data to determine the outcomes of the games, the number of players each team had on the field at the conclusion of each match, and identifying the teams that had the most victories under these conditions. This problem involves handling and interpreting event data, particularly focusing on player counts and match outcomes.

2.0.2 Solution

The process began with the loading and parsing of the dataset into a Resilient Distributed Dataset (RDD). This initial step was crucial, as it laid the groundwork for the subsequent transformation and analysis. I meticulously split each line of the dataset, representing individual match details, into distinct fields. This parsing was centered on extracting key elements such as match ID, team, and events, ensuring that the foundation of my analysis was built on accurately segregated data.

Following the data parsing, I employed a mapping strategy to transform the parsed data into a pair RDD. This was a critical stage. I implemented a nuanced logic to calculate goals and red card counts, differentiating between regular goals, own goals, and penalty goals. This careful consideration of event types was pivotal in representing each team's performance authentically and accurately.

The next phase involved aggregating this data by key. This step was essential as it combined the goal and red card data for each team in every match, providing a comprehensive view of their performance. The aggregation enabled a clear representation of each team's scoring ability and disciplinary record within the context of individual matches.

The next phase in the program's logic is determining the winning team for each match. In instances where teams end up with the same goal count, the program recognises these as draws, setting aside matches without a clear winner.

A significant part of the implementation was the extraction of winning teams along with their red card counts into a new pair RDD. This extraction was pivotal as it focused specifically on matches where teams won, despite the challenges posed by red card-induced player reductions.

I then filtered these teams to identify those with the maximum number of red cards in their wins. This filtering was a strategic move to narrow down the dataset to teams that not only won their matches but did so under notably challenging circumstances.

In the final stages of my implementation, I counted the wins for each team that met the maximum red card criteria. Using a custom comparator, I then identified the team with the most wins under these conditions.

WorldCupDriver

```
package it.unisa.hpc.worldcup;

import java.util.List;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import scala.Tuple2;
```

```
import java.util.Comparator;
import scala.Serializable;



/**
 *
 * @author chiar
 */
public class WorldCupDriver {

private static class TupleComparator implements Comparator<Tuple2<String, Integer>>, Serializable {
@Override
public int compare(Tuple2<String, Integer> t1, Tuple2<String, Integer> t2) {
    return t1._2.compareTo(t2._2);
}
}
/**
 * @param args the command line arguments
 */
public static void main(String[] args) {

if(args.length != 2){
    System.err.println("Usage: WorldCupDriver <input path> <output path>");
    System.exit(-1);
}

String inputPath = args[0];
String outputPath = args[1];
SparkConf conf = new SparkConf().setAppName("WorldCupAnalysis");
JavaSparkContext sc = new JavaSparkContext(conf);

// Carica il dataset
JavaRDD<String> lines = sc.textFile(inputPath);

JavaPairRDD<String, Tuple2<Integer, Integer>> goalsAndCardsData = lines
.mapToPair(line ->{
String fields[] = line.split(",", -1);
String match_id = fields[1];
String team = fields[2];
String events = fields[8];

int goals = 0;
int auto_goals = 0;
int red_cards = (events.contains("SY") || events.contains("R")) ? 1 : 0;
for (int i = 0; i < events.length(); i++) {
if (events.charAt(i) == 'G') {
// Assicurati che 'G' non sia preceduto da 'O' per evitare di conteggiare 'OG'
if (i == 0 || events.charAt(i - 1) != 'O') {
    goals += 1;
} else if(events.charAt(i - 1) == 'O'){
    auto_goals += 1;
}
} else if(events.charAt(i) == 'W'){
auto_goals += 1;
} else if(events.charAt(i) == 'P'){
if (i == 0 || events.charAt(i - 1) != 'M') {
goals += 1;
}
}
}
```

```
return new Tuple2<String, Tuple2<Integer, Integer>>
(match_id + "_" + team, new Tuple2<Integer, Integer>(goals - auto_goals, red_cards));
}).reduceByKey((a, b) -> new Tuple2<Integer, Integer>(a._1() + b._1(), a._2() + b._2())) ;

JavaPairRDD<String, Tuple2<String, Tuple2<Integer, Integer>>> matchResultData = goalsAndCardsData
.mapToPair(item -> {
String parts[] = item._1().split("_");
String match_id = parts[0];
String team = parts[1];

return new Tuple2<String, Tuple2<String, Tuple2<Integer, Integer>>>
(match_id, new Tuple2<String, Tuple2<Integer, Integer>>(team, new Tuple2<Integer, Integer>
(item._2()._1(), item._2()._2())));}).reduceByKey((a, b) -> {
if(a._2()._1() > b._2()._1()){
return a;
} else if (b._2()._1() > a._2()._1()){
return b;
} else{
return new Tuple2<String, Tuple2<Integer, Integer>>(null, new Tuple2(-1, -1));
}
}) ;

JavaPairRDD<String, Integer> winnersCards = matchResultData
.mapToPair(item -> {
String team = item._2()._1();
int cards = item._2()._2()._2();

return new Tuple2(team, cards);
});

JavaRDD<Integer> cards = winnersCards.values();
int max_cards = cards.top(1).get(0);

JavaPairRDD<String, Integer> teamWins = winnersCards
.filter(item -> {return (item._2() == max_cards);})
.mapToPair(item -> new Tuple2<String, Integer>(item._1(),1))
.reduceByKey((a, b) -> a + b);

TupleComparator tupleComparator = new TupleComparator();
Tuple2<String, Integer> teamWithMostWins = teamWins.max(tupleComparator);

int maxWins = teamWithMostWins._2();

JavaPairRDD<String, Integer> teamsWithMaxWins = teamWins.filter(item -> item._2().equals(maxWins));

// Visualizza tutte le squadre con il maggior numero di vittorie
teamsWithMaxWins.saveAsTextFile(outputPath);


sc.close();
}

}
```

2.1 Ouput

The output from the WorldCupDriver Spark Program is represented in the file **output.txt** in the folder **output2** for the local execution and in the **output.txt** in the folder **output7** for the cluster execution, which contain the initials of the team who has won the most matches with the fewest number of players on the field at the end of the game along with the number of matches won in this condition. The content of the file is:

```
(DEN,1)
(NIR,1)
```